

LEKTRA

High-performance Document and Image viewer that prioritizes screen space and control.

Important Links

Homepage

<https://dheerajshenoy.github.io/lektra>

Documentation

<https://lektra.readthedocs.io>

Source (Codeberg)

<https://codeberg.org/lektra/lektra>

Source (GitHub mirror)

<https://github.com/dheerajshenoy/lektra>

Note: This guide assumes you have installed **LEKTRA** and have not changed the default keybindings.

Contents

1	Getting Started	4
1.1	Opening a file	4
1.2	Tabs and Splits	4
1.3	Keyboard Navigation and Keybindings	5
1.3.1	Scrolling and Movement	5
1.3.2	Page Navigation	5
1.3.3	Search	6
1.3.4	Narrow (Focused Reading)	6
1.3.5	Zoom and View Control	6
1.3.6	Layout Modes	7
1.3.7	Reading Modes	7
1.3.8	Outline and History	7
1.3.9	Bookmarks and Marks	8
1.3.10	Annotations and Interaction Modes	8
1.3.11	Links and Actions	8

1.3.12	Editing and UI	9
1.3.13	Caret Mode	9
1.3.14	Rotation and Flip	9
1.4	Sessions	9
1.5	Portal Mode	10
1.6	File Properties	10
1.7	Copying and Exporting	10
1.8	Encrypted PDFs	10
1.9	Command Palette and Pickers	11
1.10	Updates, What's New and Support Reminders	11
2	Configuration	11
2.1	Key sections	12
2.2	Example snippet	13
2.3	Per-view and per-file-type options	13
2.4	Editor support for the config file	13
2.5	Customizing the statusbar	13
3	Commands	14
4	Lua API	15
4.1	Configuration file	15
4.2	Namespaces	15
4.3	Quick examples	16
4.4	Type stubs	17
5	AI Assistant	17
5.1	Connecting to a model	17
5.2	What it can do	18
5.3	Using the chat	18
6	LaTeX & SyncTeX	18
7	Troubleshooting	19
7.1	DjVu files don't open	19

7.2	SVGs look blurry or don't render vector graphics correctly	19
7.3	No EXIF metadata in File Properties for images	19
7.4	Where LEKTRA keeps its files	19
7.5	LEKTRA crashed	20

8	Support	20
----------	----------------	-----------

1 Getting Started

1.1 Opening a file

You can open a document in multiple ways:

- **Command line:** `lektra <path-to-file>`
- **File explorer:** Right-click a file and choose Open with lektra
- **From within lektra:** Press `o` to open the file picker

Supported formats include PDF, EPUB, XPS, CBZ, MOBI, FB2, and common image formats (JPG, PNG, APNG, WEBP, TIFF, GIF, BMP, ICO, SVG, PPM, PGM, PBM). DjVu, high-quality SVG rendering (via librsvg), and EXIF metadata are detected at runtime — install `libdjvulibre21`, `librsvg2`, and `libexif12` respectively for those features; LEKTRA continues to work without them.

You can also:

- Open multiple files at once — each opens in its own tab
- Open a file in a new window with `:open_file_new_window`
- Drag and drop files from your file manager onto the LEKTRA window
- Use `:files_recent` (or the File → Recent Files menu) to reopen a previously viewed document at the last page you were on

When `behavior.auto_reload` is enabled in `config.toml`, LEKTRA watches the open documents on disk and reloads them automatically when they change — useful for `latexmk` workflows or any tool that re-exports the PDF. With auto-reload disabled, use `:file_reload` to reload manually.

First run: On the first launch LEKTRA shows a support dialog with donation links (Ko-fi, Liberapay, GitHub Sponsors). You can reopen it any time from Help → Donate / Support or via `:donate`. LEKTRA is free forever — the dialog is entirely optional.

1.2 Tabs and Splits

LEKTRA supports tabs and splits for viewing multiple documents simultaneously.

- `o` opens a file in a new tab
- `Alt+1` ... `Alt+9` jumps directly to tab 1 through 9
- Cycling and closing tabs have no default keybinding — bind `tab_next`, `tab_prev`, `tab_close` in your config, or invoke them from the `:` command palette

Splits use Vim-style `Ctrl+W` chords:

- `Ctrl+W,s` splits horizontally (`split_horizontal`)
- `Ctrl+W,v` splits vertically (`split_vertical`)
- `Ctrl+W,h` / `Ctrl+W,j` / `Ctrl+W,k` / `Ctrl+W,l` moves focus left / down / up / right
- `Ctrl+W,c` closes the current split

Splits are independent views — each remembers its own zoom, page position, and fit mode. When `split.focus_follows_mouse` is enabled in the config, hovering over a pane also gives it focus.

Tabs across windows: You can drag a tab out of the tabbar to detach it into its own window, or drop it onto another LEKTRA window to move it there.

Selecting several tabs: `Ctrl+click` toggles a tab in the selection and `Shift+click` selects a range; selected tabs are highlighted. Right-click a selected tab for a menu for the whole group: close them, merge them into a vertical or horizontal split, move them to a new window, or save them as a session. The same operations are commands — `tabs_select_toggle`, `tabs_select_all`, `tabs_select_clear`, `tabs_close_selected`, `tabs_merge_vertical`, `tabs_merge_horizontal`, `tabs_split_out`, `tabs_move_to_window` and `tabs_save_session` — and they act on the selected tabs, or on the current tab when none is selected. A tab that holds several splits also has a Move Splits to Separate Tabs item in its menu. `:tab_rename` gives a tab a custom title.

1.3 Keyboard Navigation and Keybindings

The core philosophy of **LEKTRA** is keyboard-driven navigation (mouse support is available too). Default keybindings follow VIM conventions.

1.3.1 Scrolling and Movement

- **Vertical:** `j` scrolls down, `k` scrolls up
- **Horizontal:** `h` scrolls left, `l` scrolls right
- `Ctrl+d` / `Ctrl+u` — scroll down / up half a page

1.3.2 Page Navigation

- `Shift+j` moves to the next page
- `Shift+k` moves to the previous page
- `g,g` (chord) jumps to the first page
- `Shift+g` jumps to the last page
- `Ctrl+g` prompts for a page number and jumps to it

1.3.3 Search

- `/` opens or focuses the search bar
- `n` jumps to the next search result
- `Shift+n` jumps to the previous search result
- `:search_regex` opens search with regex mode primed
- `:search_below term` / `:search_above term` search only pages after / before the current page (one-shot; the next plain search reverts to whole-document scope)
- `:picker_annot_comment_search` finds text inside annotation comments and popups

1.3.4 Narrow (Focused Reading)

You can restrict the viewport, search, and scrolling to a subset of the document:

- `:narrow_to_region` — first enter region-selection mode (`4`) and drag out a rectangle, then run this command; only that region stays visible, the rest dims
- `:narrow_to_pages 5 10` — restrict to a page range (also accepts 5-10)
- `:narrow_to_section` — restrict to a section by outline title (fuzzy match, picker if no argument)
- `:widen_region` — exit narrow mode

While narrowed, an orange **N** badge appears in the statusbar; hover it for a reminder of how to exit. Search stays inside the narrowed range and rotation/flip remap the region correctly.

1.3.5 Zoom and View Control

- `=` zooms in, `-` zooms out, `0` resets zoom

Fit modes:

- `Ctrl+Shift+W` fit to width
- `Ctrl+Shift+H` fit to height
- `Ctrl+Shift+=` fit to window
- `Ctrl+Shift+R` toggle auto-resize

`:trim_margins` toggles trimming of blank page margins: each page is cropped to the area that holds its content, so text fills more of the screen. The content area of a page is measured in the background the first time the page is shown, so a long document settles page by page as you scroll.

1.3.6 Layout Modes

For multi-page documents you can pick how pages are arranged:

- **Single** — one page at a time
- **Continuous vertical** — pages stacked vertically (default for PDFs)
- **Continuous horizontal** — pages side by side
- **Book** — two-page facing spread with a cover offset (for magazines, novels, etc.)

Set a layout directly with one of `:layout_single`, `:layout_vertical`, `:layout_horizontal`, `:layout_book`, or use the View → Layout menu. The default at startup is controlled by `layout.mode` in `config.toml`.

1.3.7 Reading Modes

- `Ctrl+Shift+M` — toggle the menubar (menubar)
- `:fullscreen` — toggle full-screen (no default keybinding)
- `:presentation_mode` — presentation mode (hides all chrome, one page at a time)
- `:focus_mode` — focus mode (hides menubar/tabbar/statusbar while keeping the split UI)
- `:tabs`, `:statusbar` — toggle the tabbar and statusbar

1.3.8 Outline and History

- `t` opens the document outline (table of contents) overlay
- `Alt+Shift+H` opens the highlight annotation search overlay
- `Ctrl+o` goes back to the previous location in history
- `Ctrl+i` goes forward in the location history

If the PDF has no embedded outline, LEKTRA can build one from the document text by scanning font sizes: run `:generate_outline` and the result appears in the outline picker. Outlines can also be exported to (`:export_outline`) and loaded from (`:load_outline`) a portable JSON file, so you can share a hand-crafted TOC between machines.

1.3.9 Bookmarks and Marks

Two independent mechanisms for jumping around:

- **Bookmarks** are persistent, named per-document markers. Add one with `:bookmark_add`, open the picker with `:bookmarks`, or manage them from the Bookmarks menu. Bookmarks survive between sessions.
- **Marks** are single-letter jump points inside the current document, Vim-style. Press `:mark_set` then a letter to set, `:mark_goto` then a letter to jump back. `:mark_delete` clears one. Marks live only for the current session.

1.3.10 Annotations and Interaction Modes

Note: These keys toggle modes. Some modes require clicking or dragging on the page to take effect.

- `1` text-selection mode
- `2` highlight-annotation mode
- `3` rectangle-annotation mode
- `4` region-selection mode
- `5` popup-annotation mode

1.3.11 Links and Actions

- `f` link-hint visit (keyboard link navigation)
- `o` open file in a new tab
- `Ctrl+Shift+O` open file picker (`file_picker`)
- `Alt+Shift+O` recent files picker (`files_recent`)
- `Ctrl+S` save current document

With `links.hover_preview = true`, resting the cursor on an internal link shows a small picture of what it points at (a page, figure or reference) instead of a tooltip; its size and delay are `links.hover_preview_width`, `hover_preview_height` and `hover_preview_delay`.

1.3.12 Editing and UI

- `u` undo, `Ctrl+R` redo
- `v` visual-line mode
- `i` invert colors
- `Ctrl+Shift+M` toggle menubar
- `:` open command palette
- `.` repeat the last command, with the same arguments (`run_last_command`)

1.3.13 Caret Mode

`F7` (`:caret_mode`) shows a keyboard-driven text cursor, like Firefox’s “caret browsing”, for moving through the text one character at a time:

- Arrow keys or `h` `j` `k` `l` move the cursor; `Home` / `End` go to the start / end of the line
- Hold `Shift` with a movement key (or use `H` `J` `K` `L`) to extend a selection
- `y` copies the selection

1.3.14 Rotation and Flip

- `<` rotate counter-clockwise, `>` rotate clockwise
- `|` flip horizontal, `_` flip vertical

1.4 Sessions

A session is a snapshot of your open tabs, splits, and per-view state (page, zoom, layout, rotation). Sessions let you switch between reading projects without losing your place.

- `:session_save` — save the current layout to the named session (or the active one)
- `:session_save_as` — save under a new name
- `:session_load` — pick a session from the list and restore it

Sessions live under `$HOME/.local/share/lektra/sessions/` (Linux/macOS) as JSON files, one per session.

1.5 Portal Mode

Portal mode links two views to the same document so that scrolling one drives the other — useful for cross-referencing figures with body text, or keeping a table of contents pane in sync with the main view. Trigger with `:portal`.

1.6 File Properties

`:file_properties` (or File → File Properties) opens the properties dialog. For PDFs it shows the info dictionary (title, author, producer, creation date, PDF version, encryption status, page count). For images it shows width, height, format, animation status, and — when `libexif` is installed — full EXIF metadata (camera model, lens, ISO, shutter, aperture, GPS, date taken).

1.7 Copying and Exporting

- `:selection_copy` — copy the current text selection to the clipboard
- `:export_pages` — save pages as images (PNG by default), PDF, SVG, text, HTML, CBZ and more. A dialog asks which pages (for example 1-5,8,10-, all, odd, current) and the format, then where to save. For PDF, text and HTML a checkbox chooses between one file with all the pages and one file per page (pictures and SVG are always one per page); files made per page get the page number added to the name (out-2.png). Give a path, pages and a resolution to skip the dialogs, e.g. `:export_pages /tmp/p.png 3-5 200`, and add `split` to make one file per page (`:export_pages /tmp/p.pdf 3-5 150 split`). From Lua: `view:export_pages("/tmp/p.png", "3-5", { dpi = 200 }), with split = true` for one file per page
- `:copy_page_image` — copy the current page as an image (context menu also has “Copy Region as Image” when a region is selected, with an optional custom DPI for vector documents)
- **Drag an image out:** click and drag an image from a PDF page onto another application or a file manager to save or paste it as a PNG, at its native resolution
- `:export_outline` — write the document outline (real or generated) to JSON
- `:file_save_as` — save the current PDF to a new location (including a copy of any annotations you have added)

1.8 Encrypted PDFs

Password-protected PDFs prompt for a password on open. You can also encrypt or decrypt a PDF from within LEKTRA:

- `:file_encrypt` — add a password to the current document
- `:file_decrypt` — remove the password (requires the current one)

1.9 Command Palette and Pickers

Press `:` to open the command palette — a fuzzy-matching prompt that runs any LEKTRA command by name. Beyond commands, several dedicated pickers exist:

- `:command_palette` — list every command with its description
- `:file_picker` — open a file
- `:files_recent` — jump to a recently opened file, restored to its last page
- `:picker_outline` — outline / TOC
- `:bookmarks` — bookmarks
- `:picker_highlight_search` — search across highlight annotations

1.10 Updates, What's New and Support Reminders

Now and then a slim banner appears at the bottom of the window, above the statusbar. It never blocks the document, and each one has a close button.

- **A newer release.** About once a day LEKTRA asks the GitHub releases page for the latest version (one request; nothing about you or your documents is sent). If a newer one exists, the banner offers Release notes and Skip this version. `:check_for_updates` (or Help → Check for Updates) checks right away and always tells you the answer. Turn the daily check off with `[updates] check = false`.
- **What's new.** `:whats_new` (or Help → What's New) lists the changes of the version you are running. With `[updates] whats_new = true` a banner offers the list once after each update (off by default).
- **A support reminder.** After a few weeks of use, and at most twice a year, a banner mentions that LEKTRA can be supported. Don't ask again ends it for good, and `[donate] reminders = false` turns it off in the config.

2 Configuration

LEKTRA can be configured using **TOML** or **Lua**. Both files are optional and both can be present; the TOML file is applied first, and the Lua file is sourced afterwards so it can override any TOML setting or add runtime behaviour.

Config file locations:

Linux / macOS

```
$HOME/.config/lektra/config.toml
```

```
$HOME/.config/lektra/init.lua
```

Windows

%APPDATA%\lektra\config.toml

%APPDATA%\lektra\init.lua

Run `:open_config` to open whichever config file(s) exist — a chooser appears if both are present.

2.1 Key sections

[window]

Window size, fullscreen, menubar visibility, title format, accent color.

[page]

Page foreground and background colors.

[statusbar]

Visibility, padding, which modules to show and where (layout), and per-module options in `[statusbar.components.*]`.

[split]

Split behaviour — focus follows mouse, dim inactive panes, etc.

[portal]

Picture-in-picture portal overlay settings.

[annotations.highlight] / [annotations.rect] / [annotations.popup]

Per-annotation-type appearance (colours, hover glow, comment marker size).

[thumbnail_panel]

Thumbnail strip width, page numbers, and scrollbar visibility.

[keybindings]

Override any keybinding by specifying `action = "Key"` or `action = [ "Key1", "Key2" ]`.

[mousebindings]

Mouse button bindings, similar to keybindings.

[reflow]

Font family, font size and line spacing for EPUB and other reflowable documents.

[filetype.<type>.<section>]

Overrides for one kind of document, e.g. `[filetype.epub.layout]`.

[updates]

The daily update check and the “what’s new” banner.

[donate]

The support reminder.

[llm_view]

The AI assistant panel (see the [AI Assistant](#) section).

2.2 Example snippet

```
[window]
menubar = false
accent = "#3daee9FF"

[statusbar]
visible = true
padding = 4
layout = [ "filename", "|", "page", "zoom", "|", "progress", "mode" ]

[page]
bg = "#1e1e2e"
fg = "#cdd6f4"
```

2.3 Per-view and per-file-type options

Every view and split keeps its own copy of the view settings (page colours, zoom, layout, rendering, scrollbars, selection, search, annotations, links, behaviour), like Vim's window-local options. A split starts with the options of the view it was split from. From Lua, `view:opt()` changes one view, while `lektra.opt` sets the global default and the current view.

In `config.toml` you can override those settings for one kind of document, for example:

```
[filetype.epub.layout]
mode = "single"

[filetype.pdf.behavior]
auto_reload = true
```

2.4 Editor support for the config file

LEKTRA ships a JSON Schema for `config.toml` (installed to `$prefix/share/lektra/lektra-config.schema.json`). With a TOML language server such as [Taplo](#) it gives completion, a description of each option on hover, and warnings for typos and wrong values. The default config starts with a `#:schema` line that points at it. A setting with the wrong type (or an invalid colour) is also reported when LEKTRA starts, with its line number.

2.5 Customizing the statusbar

The statusbar is made of **modules**: `session`, `filename`, `page`, `zoom`, `progress`, `mode`, `portal` and `narrow`. `[statusbar].layout` says which are shown and where; a module that is not listed is not

shown. padding is either one number (all four sides) or [left, top, right, bottom].

Items go from left to right and the free space goes to the **gaps**: "|" is a gap that takes all of it, { stretch = 2 } a gap with twice the share of another one, and { spacer = 12 } a gap of 12 pixels. So "left, centre and right" is just two gaps, and any other arrangement works as well.

```
[statusbar]
padding = 4

# page and zoom in the middle, the mode on the right
layout = [ "filename", "|", "page", "zoom", "|", "mode" ]

# everything on the right, with separators
layout = [ "|", "filename", { text = "•" }, "page", { text = "•" },
          "zoom", "mode" ]

# gaps with different weights, and a fixed gap
layout = [ "mode", { spacer = 16 }, "filename",
          { stretch = 1 }, "page", { stretch = 3 }, "zoom" ]

# modules at fixed positions along the bar (0 to 1)
layout = [ "filename",
          { module = "page", at = 0.25, anchor = "center" },
          { module = "mode", at = 1.0, anchor = "right" } ]

# several rows: a list of lists
layout = [ [ "filename", "|", "session" ],
          [ "page", "|", "zoom", "progress", "mode" ] ]
```

A module can also be a table with more options: stretch (takes a share of the free space), min_width and max_width (pixels), margin (a number or [left, right]), and align ("left", "center" or "right" inside its slot). With at (the position along the bar, 0 to 1) and anchor (which edge of the item is put there) it is placed at a fixed spot instead of in the flow. { text = "..." } shows a piece of text. Entries that are not valid are skipped and reported when LEKTRA starts. From Lua: lektra.opt.statusbar.layout = { ... }, which takes effect at once.

For the full list of options see the [configuration reference](#) on the homepage.

3 Commands

Press  to open the command palette and run any command by name. Commands cover opening files, navigation, zoom, annotations, sessions, and more.

The full command reference is on the [commands page](#) of the homepage.

4 Lua API

LEKTRA embeds a bundled **LuaJIT** interpreter, linked statically and enabled by default (build with `-DWITH_LUA=OFF` to leave it out), so no Lua installation is needed. Scripts run as Lua 5.1 with LuaJIT's extensions: `goto` and the `bit` library are available, but Lua 5.3 features such as `//`, `utf8` and `string.pack` are not. All APIs live under the global `lektra` table.

4.1 Configuration file

Place your Lua init file at `$HOME/.config/lektra/init.lua` (Linux / macOS) or `%APPDATA%\lektra\init.lua` (Windows). It is sourced at startup after the TOML config is applied.

4.2 Namespaces

`lektra.view` Access the current view, get one by id, or list all views; the view object exposes navigation, zoom, rotation, layout, search, text (`page_text` gives the text of any page), links, scrolling, outline, and per-view event listeners (`register`, `register_once`, `unregister`, `clear_listeners`, `register_context_menu`).

`lektra.opt` Read/write runtime options (fit mode, layout mode, DPR, statusbar layout, ...); `view:opt()` does the same for one view.

`lektra.cmd` `register`, `unregister`, `execute`, `list`, `alias` — expose or invoke lektra commands from Lua.

`lektra.keymap` `set`, `get` — bind keys to Lua functions at runtime.

`lektra.mousemap` `set` — bind mouse buttons to Lua functions.

`lektra.event` `register`, `unregister`, `once`, `count`, and the `EventType` enum — global event listeners. Events include `OnFileOpen`, `OnFileClose`, `OnFileSaved`, `OnPageChanged`, `OnZoomChanged`, `OnModeChanged`, `OnTabAdded`, `OnTabRemoved`, `OnSessionLoaded`, `OnBookmarkAdded`, the search events and more.

`lektra.ui` `message`, `messagebox`, `input`, `menu`, `picker`, `file_dialog`, `color_dialog`.

`lektra.bookmarks` `list` and `add` — list the bookmarks, or add one (at the current location by default).

`lektra.tabs` `list`, `count`, `current`, `close`, `goto`, `first`, `last`, `next`, `prev`, `move_left`, `move_right`, `get_id`, `rename`, and the multi-select helpers `selected`, `select`, `select_all`, `clear_selection`, `close_selected`, `merge`, `split_out`, `move_to_window`, `save_session`.

`lektra.sessions` `list`, `exists`, `load`, `delete`.

`lektra.recent_files` `list` — the recently opened files.

`lektra.window open` — open a file in a new window.

`lektra.clipboard get, set` — read and write the text on the clipboard.

`lektra.capabilities image, synctex, djvu` — what this build supports.

`lektra.timer new` — create one-shot / repeating timers driven by the Qt event loop.

`lektra.utils print, open_url, platform`.

`lektra.version major, minor, patch, full` — LEKTRA's version at runtime.

4.3 Quick examples

Bind a key to a Lua function. First register a lektra command whose implementation is your Lua callback, then bind keys to that command:

```
lektra.cmd.register("say_page", function()
    local v = lektra.view.current()
    lektra.ui.message("Page: " .. v:pageno())
end, "Print current page in the message bar")

lektra.keymap.set("say_page", { "<C-e>" })
```

React to a page change event:

```
lektra.event.register(lektra.event.EventType.OnPageChanged, function()
    local v = lektra.view.current()
    print("Now on page", v:pageno())
end)
```

Open a file in the current view:

```
local v = lektra.view.current()
v:open("/path/to/document.pdf")
```

Run a command in the background and get called back:

```
lektra.job.async({"sleep", "3"}, { timeout = 10 }, function(r)
    lektra.ui.message(r.ok and "Finished" or ("Failed: " .. tostring(r.error)))
end)
-- the script (and the window) carry on immediately
```

Copy the text of a page and react to new bookmarks:

```
lektra.cmd.register("copy_page_text", function()
    local v = lektra.view.current()
    lektra.clipboard.set(v:page_text(v:pageno()))
    lektra.ui.message("Copied the text of page " .. v:pageno())
end, "Copy the text of the current page")

lektra.event.register("OnBookmarkAdded", function(b)
    lektra.ui.message("Bookmarked page " .. b.pageno)
end)
```

Change the statusbar layout while running:

```
lektra.opt.statusbar.padding = 4
lektra.opt.statusbar.layout = { "filename", "|", "page", "zoom", "|", "mode" }
```

4.4 Type stubs

LSP-friendly type stubs for the Lua API are installed at `$prefix/share/lektra/lua/` (Linux/macOS). Point your Lua language server at this directory for autocompletion. Every option in the stubs is optional, so assigning a partial table such as `lektra.opt.statusbar = { padding = 4 }` does not raise a warning.

```
// .luarc.json example
{
    "workspace.library": ["/usr/share/lektra/lua"]
}
```

For the complete API reference with all methods, types, and event names, see the [Lua API](#) page on the homepage.

5 AI Assistant

LEKTRA has an optional chat panel for a language model. It is built only with `-DWITH_LUA=ON -DWITH_LLM_SUPPORT=ON` (the second option is off by default). Toggle the panel with `:llm_view`, or open it at startup with `llm_view.show_at_startup = true`.

5.1 Connecting to a model

The panel talks to any server with an OpenAI-compatible chat endpoint: a local one (Ollama, llama.cpp, LM Studio) or a hosted service. The indicator above the chat shows whether the endpoint is reachable; hover it for the model, provider and connection details.

```
[llm_view]
api_url = "http://localhost:11434/v1/chat/completions"  # the default: Ollama
model   = "qwen3:8b"
api_key = ""                                           # empty for local models
extra_body = { stream = true }                       # extra fields sent with every request
```

Everything you type, and any image you attach, is sent to the configured endpoint; the API key is read from `config.toml` (or `init.lua`), so keep that file private.

5.2 What it can do

- **Answer questions** about LEKTRA, in Markdown, with Lua code highlighted and every code block collapsible. LaTeX maths is drawn: $\dots$ inside a line and $\dots$ on its own line.
- **Control LEKTRA.** The assistant acts through tools: `run_command` for a single command, `run_lua` for a Lua script (read state, change options, several steps), and `lookup_api` to read the Lua API when it needs it, which keeps each request small. Every action asks for confirmation with Run or Skip, unless you set `llm_view.auto_run = true`. Scripts run in a restricted environment (no `io`, `os.execute`, `require`, `load` or `ffi`) with a time limit, and their result goes back to the model.
- **See images.** Attach one with the + button, by pasting, or by dropping a file: the current page, a region of the page you select, or an image file. Vision-capable models can describe or analyse it.

Models that do not support tool calling (many local ones) work with `llm_view.tools = false`: the whole Lua API is then sent with every request and the assistant replies with a Lua code block that you run with a button.

5.3 Using the chat

- `Shift+Enter` sends the message (plain `Enter` makes a new line). While a reply is coming in, the send button becomes a **Stop** button (so does `Esc`); what had arrived stays in the chat.
- Hover a message and click the clipboard icon beside it to **copy** it as written, with its LaTeX and code blocks as source.
- Chats are saved and can be reopened from the History menu; New chat starts a fresh one. Turn saving off with `llm_view.save_history = false`.
- `llm_view.font_size` sets the size of the message text.

6 LaTeX & SyncTeX

LEKTRA builds with SyncTeX support enabled by default. If your PDF was compiled with SyncTeX information (`-synctex=1` with `pdflatex`/`xelatex`/`lualatex`, or the equivalent flag in `latexmk`), you

can:

- **Forward-search** from your editor into LEKTRA — jump from a source line to the corresponding location in the PDF. Editor integration snippets for Neovim, VS Code, Emacs, and TeXstudio are on the homepage.
- **Reverse-search** from LEKTRA back to your editor — click while holding the SyncTeX modifier (Ctrl by default) and LEKTRA runs your configured editor command with the source file and line.

Enable `behavior.auto_reload` in `config.toml` and re-running `latexmk` on a source change reloads the PDF automatically without losing your page position or zoom. Otherwise use `:file_reload` to reload manually.

7 Troubleshooting

7.1 DjVu files don't open

LEKTRA loads `libdjvulibre.so.21` at runtime. On Debian/Ubuntu install `libdjvulibre21`; on Arch install `djvulibre`; on Fedora `djvulibre`. No rebuild is needed — restart LEKTRA and DjVu will work.

7.2 SVGs look blurry or don't render vector graphics correctly

LEKTRA prefers `librsvg + libcairo` for SVG rendering; if either is missing it falls back to Qt's own SVG renderer, which handles a smaller subset of the spec. Install `librsvg2` (SONAME `.so.2`) for full quality.

7.3 No EXIF metadata in File Properties for images

Install `libexif` (SONAME `.so.12`). LEKTRA discovers it at runtime — no rebuild required.

7.4 Where LEKTRA keeps its files

Linux / macOS:

```
$HOME/.config/lektra/  
  config.toml, init.lua  
$HOME/.local/share/lektra/  
  recent files DB, sessions, bookmarks  
$HOME/.local/share/lektra/crashes/  
  crash logs (crash_latest.log)
```

`$HOME/.local/share/lektra/sessions/`

JSON session files

`$HOME/.local/share/lektra/llm_chats/`

saved AI assistant chats (JSON)

`$HOME/.local/share/lektra/app_state.json`

when LEKTRA was first used, how often it was started, and which banners were shown

`$HOME/.cache/lektra/microtex/`

fonts for the maths in assistant replies (unpacked on first use; safe to delete)

Windows (paths resolved via Qt's `QStandardPaths`; `%APPDATA%` is typically `C:\Users\<user>\AppData\Roaming`)

`%APPDATA%\lektra\`

`config.toml`, `init.lua`, recent files DB, sessions, bookmarks

`%APPDATA%\lektra\sessions\`

JSON session files

`%TEMP%\lektra_crash.log`

latest crash log

Windows collapses the config and app-data locations because LEKTRA does not set a Qt organization name, so both `QStandardPaths::AppConfigLocation` and `AppDataLocation` resolve under the same `%APPDATA%\lektra\` directory.

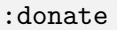
7.5 LEKTRA crashed

A crash dialog appears with the log and a one-click “Report on GitHub” button that pre-fills an issue. You can also copy the log to the clipboard from that dialog, or find it later at `crash_latest.log` in the crashes directory. Debug and `RelWithDebInfo` builds include function names and file/line numbers in the trace; release builds show addresses only — running `addr2line -e /usr/bin/lektra` on those addresses resolves them if the binary keeps symbols.

8 Support

LEKTRA is developed by Dheeraj Vittal Shenoy and contributors, and is free and open-source software that will stay free forever. If it has been useful to you, consider supporting continued development through one of:

- Ko-fi — <https://ko-fi.com/dheerajshenoy>
- Liberapay — <https://liberapay.com/dheerajshenoy>
- GitHub Sponsors — <https://github.com/sponsors/dheerajshenoy>

Bug reports and feature ideas are welcome at the issue tracker on Codeberg (<https://codeberg.org/lektra/lektra>) or on GitHub (<https://github.com/dheerajshenoy/lektra>).  inside LEKTRA reopens the support dialog at any time. The occasional support banner can be turned off with `[donate] reminders = false`.